Modelo de Aula Invertida

Tema: Ordenamiento Básico (Algoritmos Cuadráticos)

Evidencia: Reporte comparativo de algoritmos de ordenamiento básico

Integrantes:

- Jorge Eduardo Lopez Garcia
- Octavio Sebastian Parra Cajero

COMPRENDE

En esta etapa se analiza cómo funcionan tres algoritmos básicos de ordenamiento:

- **Bubble Sort**
- **Selection Sort**
- **Insertion Sort**

1. Funcionamiento paso a paso (ejemplo)

Arreglo inicial:
[5, 3, 4, 1, 2]

Bubble Sort (idea general):
Compara pares de elementos vecinos y los intercambia si están en el orden incorrecto.
Después de varias pasadas, los valores grandes “suben” al final.

Selection Sort:
Busca el valor más pequeño y lo coloca en la primera posición, luego repite con el resto.

Insertion Sort:
Toma un elemento y lo inserta en la posición correcta dentro de la parte ya ordenada.

2. Invariantes de ciclo (idea clave)

- **Bubble Sort:**
  Después de cada pasada, el último elemento ya está ordenado.

- **Selection Sort:**
  La parte izquierda del arreglo siempre está ordenada.
- **Insertion Sort:**
  Los elementos anteriores siempre están ordenados.

3. Complejidad

- **Bubble Sort:**
  - Peor caso: $O(n^2)$
- **Selection Sort:**
  - Siempre: $O(n^2)$
- **Insertion Sort:**
  - Peor caso: $O(n^2)$
  - Mejor caso: $O(n)$ (cuando ya está ordenado)

4. Estabilidad

- **Estables:**
  - Bubble Sort
  - Insertion Sort
- **No estable:**
  - Selection Sort

APLICA

Implementación en JavaScript

```
// Bubble Sort
function burbuja(arreglo) {
  for (let i = 0; i < arreglo.length; i++) {
    for (let j = 0; j < arreglo.length - 1; j++) {
```

```
        // DECISIÓN: comparar elementos vecinos
        if (arreglo[j] > arreglo[j + 1]) {

          // DECISIÓN: intercambiar si están desordenados
          let temp = arreglo[j];
          arreglo[j] = arreglo[j + 1];
          arreglo[j + 1] = temp;
        }
      }
    }
    return arreglo;
}

// Selection Sort
function seleccion(arreglo) {
    for (let i = 0; i < arreglo.length; i++) {
      let minimo = i;

      for (let j = i + 1; j < arreglo.length; j++) {

        // DECISIÓN: encontrar el menor
        if (arreglo[j] < arreglo[minimo]) {
          minimo = j;
        }
      }

      // DECISIÓN: intercambiar con la posición actual
      let temp = arreglo[i];
      arreglo[i] = arreglo[minimo];
      arreglo[minimo] = temp;
    }
    return arreglo;
}

// Insertion Sort
function insercion(arreglo) {
    for (let i = 1; i < arreglo.length; i++) {
      let actual = arreglo[i];
```

```
    let j = i - 1;

    // DECISIÓN: mover elementos mayores hacia la derecha
    while (j >= 0 && arreglo[j] > actual) {
      arreglo[j + 1] = arreglo[j];
      j--;
    }

    // DECISIÓN: insertar en la posición correcta
    arreglo[j + 1] = actual;
  }
  return arreglo;
}
```

Benchmark básico

```
function generarArreglo(n, tipo) {
  let arr = [];

  for (let i = 0; i < n; i++) {
    arr.push(Math.floor(Math.random() * 1000));
  }

  if (tipo === "ordenado") return arr.sort((a, b) => a - b);
  if (tipo === "inverso") return arr.sort((a, b) => b - a);
  return arr;
}

function medirTiempo(funcion, arreglo) {
  let inicio = Date.now();
  funcion([...arreglo]);
  let fin = Date.now();
  return fin - inicio;
}
```

Pruebas con tamaños:

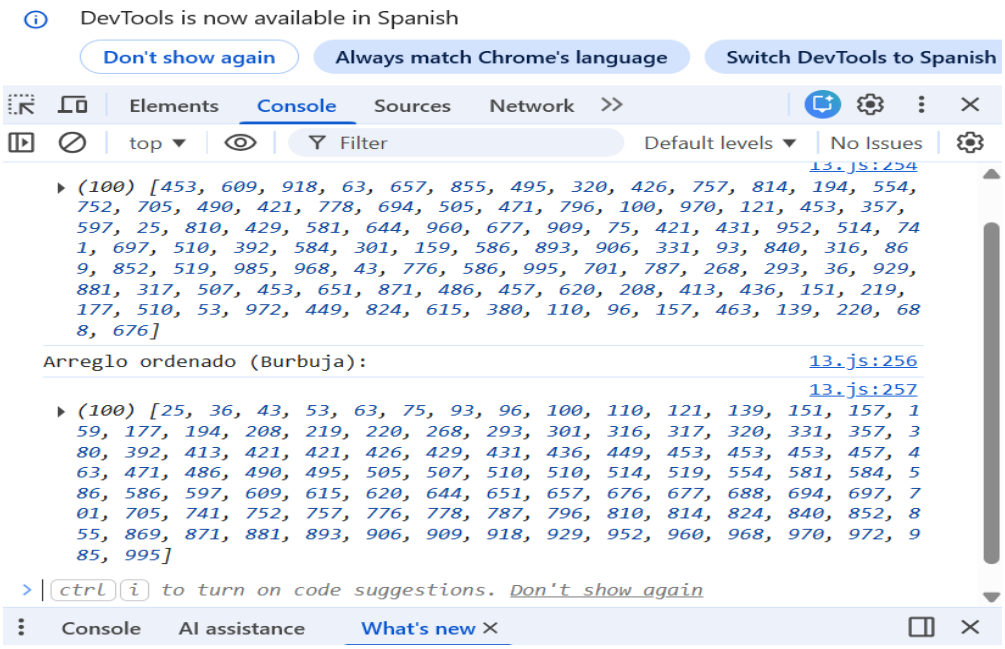
- n = 100

- n = 1000
- n = 10000

Tipos:

- Ordenado
- Inverso
- Aleatorio

```javascript
function generarArreglo(n,tipo) {
    let arr = [];

    for (let i = 0; i < n; i++) {
        arr.push(Math.floor(Math.random() * 1000));
    }

    if (tipo === "ordenado") return arr.sort((a, b) => a - b);
    if (tipo === "inverso") return arr.sort((a, b) => b - a);
    return arr;
}

function medirTiempo(funcion, arreglo) {
    let inicio = Date.now();
    funcion([...arreglo]);
    let fin = Date.now();
    return fin - inicio;
}

let arreglo = generarArreglo(100, "aleatorio");

console.log("Arreglo original:");
console.log(arreglo);

console.log("Arreglo ordenado (Burbuja):");
console.log(burbuja([...arreglo]));
```



REFLEXIONA

- **¿Por qué Insertion Sort es O(n) en mejor caso?**
  Porque si el arreglo ya está ordenado, no necesita hacer intercambios, solo una pasada.
- **¿Cuándo usar Selection Sort sobre Bubble Sort?**
  Cuando quieres menos intercambios, ya que Selection Sort hace menos cambios en memoria.
- **Comparación teoría vs práctica:**
  La teoría indica que todos son $O(n^2)$, pero en la práctica:
  - Insertion Sort es más rápido en arreglos pequeños o casi ordenados
  - Bubble Sort suele ser el más lento

VALIDA

Checklist de pruebas:

- Arreglo vacío → []
- Un elemento → [5]

- Ya ordenado → [1,2,3,4]
- Inverso → [4,3,2,1]
- Con duplicados → [2,2,1,1]

Se debe verificar:

- Que el resultado esté ordenado
- Que los tiempos coincidan con la teoría

Tabla Comparativa de Algoritmos de Ordenamiento Básico

Característica	**Bubble Sort**	**Selection Sort**	**Insertion Sort**
Idea principal	Intercambia elementos adyacentes repetidamente	Selecciona el mínimo y lo coloca en su lugar correcto	Inserta cada elemento en su posición dentro de un subarreglo ordenado
Complejidad(mejor caso)	O(n) (con optimización)	$O(n^2)$	O(n)
Complejidad(peor caso)	$O(n^2)$	$O(n^2)$	$O(n^2)$
Complejidad promedio	$O(n^2)$	$O(n^2)$	$O(n^2)$

Número de intercambios	Alto	Bajo (máximo n swaps)	Medio (más desplazamientos que swaps)
Estabilidad	Sí	No	Sí
Uso de memoria	O(1) (in-place)	O(1) (in-place)	O(1) (in-place)
Adaptativo	Sí (si se optimiza)	No	Sí
Mejor escenario	Arreglos casi ordenados	Cuando se desea minimizar swaps	Arreglos pequeños o casi ordenados
Peor escenario	Arreglos inversos	Cualquier caso (siempre igual)	Arreglos inversos
Ventaja principal	Fácil de entender e implementar	Pocos intercambios	Muy eficiente en casos favorables
Desventaja principal	Muy ineficiente en grandes datos	No es estable y siempre es lento	Ineficiente en grandes volúmenes

Conclusión:

- **Todos ordenan correctamente ✓**
- **Insertion Sort** es el más eficiente en la práctica cuando los datos ya están ordenados o casi ordenados
- **Selection Sort** reduce intercambios, pero no mejora el tiempo
- **Bubble Sort** es el menos eficiente, aunque es fácil de entender

PROFUNDIZA

Estos algoritmos tienen una gran limitación: Son muy lentos para arreglos grandes ($O(n^2)$)

Esto motiva el uso de algoritmos más eficientes basados en:

- **Divide y vencerás**

Ejemplos futuros:

- Merge Sort
- Quick Sort

Estos reducen el tiempo a **O(n log n)**, lo que los hace mucho más rápidos en grandes volúmenes de datos.